

UNREAL ENGINE 5 · C++ · WAVE DEFENSE

NEON DRIFT

상세 구현 설계서

UE 5.8 C++ 아케이드 웨이브 디펜스 — 3일 스프린트 프로젝트

Implementation Design Document

Context

`NEON_DRIFT_DETAILED_PROMPT.md` 의 기획을 실제 구현 가능한 기술 설계로 구체화한다. 현재 프로젝트는 **빈 UE 5.8 C++ 블랭크 템플릿**이다 (게임 코드/콘텐츠/맵 없음, 기본 모듈 `NEONDRIFT` + `EnhancedInput` 만 활성). 따라서 모든 게임플레이를 처음부터 C++로 작성한다.

확정된 설계 결정:

- **UI**: 순수 C++ `AHUD::DrawHUD()` Canvas 드로잉. UMG 위젯 에셋 없음.
- **씬 구성**: 클래스는 모두 에디터 배치 가능한 Actor. 사용자가 레벨에 `Base/SpawnPoint/AutoTurret/PlayerStart`를 배치하고 `GameMode`를 지정. 블록/몬스터/슬래그는 런타임 스폰.
- **빌드/검증**: CLI에서 UBT(UE_5.8)로 컴파일해 코드 오류를 잡고, 에디터 PIE로 실행 검증.
- **포탑 모델**: 기체 무기(즉시 조준) + 기지 수동 포탑(회전속도 제한, 강화 핵심) + 자동 포탑 N개 공존.
- **엔진**: UE 5.8 (uproject/Target.cs 기준), C++20, BuildSettingsVersion.V7.

엔진 미설정 사항(에셋이라 CLI 불가) → 사용자 수동 1회 작업으로 분리(아래 "Day 0" 참조).

아키텍처 개요

<code>UNeonGameInstance</code>	영구 강화 상태(웨이브 간/리스트ार्ट 생존), 파생 스탯 계산
<code>ANeonGameMode</code>	게임 루프 상태머신 + 웨이브 매니저 + 자원 카운트 + 강화 적용
<code>ANeonPlayerController</code>	<code>EnhancedInput</code> 코드 구성, 기체 방위, 상점 입력 라우팅
<code>ANeonHUD</code>	Canvas 드로잉(자원/기지HP/웨이브/페이지/상점/게임오버)
<code>APlayerShip</code>	기체: 비행(중력·관성), 기체무기(즉시조준 연사), 자석 수집
<code>ABase</code>	기지: HP, 네온/경고 색, 피격 플래시, 파괴→게임오버
<code>AManualTurret</code>	기지 중심: 플레이어 조준 추종(회전속도 제한), 자동연사
<code>AAutoTurret</code>	기지 주변 배치: 최근접 몬스터 자동 타겟·연사, 개수 강화
<code>AMonster</code>	큐브: 기지 직진, 도달 시 DPS, 피격 사망→웨이브 통지
<code>AResourceBlock</code>	금/철/다이아: 공격력 게이트, 파괴 시 슬래그 드롭
<code>AResourceShard</code>	부스러기: 자석 범위 내 호밍→수집
<code>ASpawnPoint</code>	배치 마커: 방위(N/E/S/W) 태그, 웨이브 매니저가 조회
데이터 USTRUCT: <code>FShipStats</code> , <code>FTurretStats</code> , <code>FUpgradeState</code> , <code>FWaveDef</code> , <code>FBlockDef</code> , <code>FUpgradeDef</code> (코드 인라인 테이블, <code>GameMode</code> 에 UPROPERTY 노출)	
공용 인터페이스: <code>IDamageable</code> (<code>TakeHit(float Damage, int32 AttackerPower)</code>)	

파일 배치: 전부 `Source/NEONDRIFT/` 아래 (`Public/`·`Private/` 분리 없이 평면 — 프로토타입 단순화).

`NEONDRIFT.Build.cs` 에 `EnhancedInput` 은 이미 존재. 추가 의존성 불필요(Canvas는 Engine 포함).

핵심 클래스 상세 (주요 변수 · 메서드)

IDamageable (NeonTypes.h, UINTERFACE)

- `virtual void TakeHit(float Damage, int32 AttackerPower) = 0;`
- 블록/몬스터/기지가 구현. 무기 라인트레이스가 맞은 액터에 인터페이스 캐스트로 호출.

UNeonGameInstance : UGameInstance

- `FUpgradeState Upgrades;` — 각 강화 항목의 현재 단계(레벨) 정수 맵/구조체.
- `FShipStats GetShipStats() const;` / `FTurretStats GetManualTurretStats() const;` / `FTurretStats GetAutoTurretStats() const;` / `int32 GetAutoTurretCount() const;` — 기본값 + 강화 단계로 파생 스탯 계산(단일 진실 공급원).
- `bool TryPurchase(EUpgradeId Id, int32& InOutResources);` — 비용 차감 + 단계 증가(상한 체크).
- `void ResetRun();` — 신규 게임 시작 시 초기화.

ANeonGameMode : AGameModeBase

- 상태: `enum class EGamePhase { PreWave, Gather, Combat, Shop, GameOver, Victory }`
- 변수: `EGamePhase Phase;` `int32 WaveIndex;` `int32 Resources;` `float GatherTimer;` `int32 AliveMonsters;` `int32 SpawnedThisWave;` `TArray<FWaveDef> WaveTable;` `TArray<FBlockDef> BlockTable;` `TArray<FUpgradeDef> UpgradeTable;` (셋 다 `UPROPERTY(EditAnywhere)` → 에디터 튜닝 가능, 기본값은 `BeginPlay`에서 코드로 채움)
- 배치 액터 참조: `ABase* Base;` `TArray<ASpawnPoint*> SpawnPoints;` `TArray<AAutoTurret*> AutoTurrets;` `AManualTurret* ManualTurret;`

메서드:

- `BeginPlay()` — `TActorIterator` 로 배치 액터 수집, `Base->OnDestroyed` 바인드, 리소스 필드 스폰 (`SpawnResourceField()`), `EnterPhase(PreWave)`.
- `EnterPhase(EGamePhase)` — 전이 + HUD 상태 갱신.
- `Tick()` — Gather 단계 타이머(60s, 0 도달 또는 Ready 입력 → Combat), Combat 단계 스폰 펌프 + `AliveMonsters==0 && SpawnedThisWave==total` → 웨이브 클리어.
- `RequestStartWave()` — Gather→Combat (PlayerController가 호출).
- `SpawnResourceField()` — BlockTable 비율로 튜닝 가능 영역(`FBox SpawnBounds`, `EditAnywhere`) 내 무작위 위치에 `AResourceBlock` 스폰.
- `SpawnWaveTick(float Dt)` — 현재 웨이브 def의 입구별 간격으로 `AMonster` 스폰.
- `OnMonsterKilled()` / `OnMonsterReachedBase()` — 카운터 갱신.
- `AddResources(int32)` — 슬래그 수집 시 호출, HUD 갱신.
- `OnWaveCleared()` → Shop / `OnBaseDestroyed()` → GameOver / 웨이브5 클리어 → Victory.
- `ApplyUpgrade(EUpgradeId)` — GameInstance.TryPurchase, 성공 시 기체/포탑 스탯 재적용.
- `ContinueToNextWave()` — Shop→PreWave(WaveIndex++).

APlayerShip : APawn

- 컴포넌트: `UStaticMeshComponent* Body` (Engine Cube), `USpringArmComponent* SpringArm`, `UCameraComponent* Camera`, `USceneComponent* Muzzle`, `UNeonTrailComponent* Trail` (폴리시).
- 스탯: `FShipStats Stats`; (AttackDamage, FireRate, MagnetRadius, MaxSpeed, MaxHP, CurrentHP) — `GameInstance`에서 받아 적용.
- 비행 상태: `FVector Velocity`; `float Gravity=980`; `float Accel`; `float LinearDrag`;

메서드:

- `SetupPlayerInputComponent()` — EnhancedInput 액션 바인드(IMC/IA는 PlayerController가 코드 생성).
- `Move(FVector2D)` / `VerticalThrust(float)` / `Look(FVector2D)` — 입력 핸들러.
- `Tick()` — 가속·중력·드래그 적분, `MaxSpeed` 클램프, `AddActorWorldOffset(sweep)`; Camera FOV를 속도 비례로 70→110 보간; `CollectShards()`.
- `StartFire()/StopFire()` — `bFiring` 토글, `FireTimerHandle` 로 1/FireRate 주기 `FireOnce()`.
- `FireOnce()` — Muzzle 전방 라인트레이스, `IDamageable::TakeHit(AttackDamage, AttackPower)`, 네온 빔 FX, 반동/머즐 플래시.
- `CollectShards()` — `MagnetRadius` 구체 오버랩 → 각 `AResourceShard::AttractTo(this)`.
- `TakeHit()` — 몬스터 충돌/기지 폭발 피해, CurrentHP 0 → 게임오버 통지.

ABase : AActor (IDamageable)

- `UStaticMeshComponent* Mesh`; `UMaterialInstanceDynamic* MID`;
- `float MaxHP=5`, `CurrentHP`; `float DamagePerSecond=1`; `float HitFlash`;
- `OnBaseHPChanged OnBaseHPChanged`; (멀티캐스트, HUD 구독) `FSimpleDelegate OnDestroyed`;
- `TakeHit(Damage, _)` — CurrentHP 감소, HitFlash=1로 세팅(빨강), $HP \leq 25\%$ → MID 색 주황 경고, 0 → `OnDestroyed.Execute()`.
- `Tick()` — HitFlash 감쇠 보간으로 색 복귀, 네온 경계 펄스.

AManualTurret : AActor

- `UStaticMeshComponent* Base/Barrel`; `FTurretStats Stats`; (AttackDamage, FireRate, RotationSpeed)
- `FRotator DesiredAim`; (PlayerController가 기체 조준값 전달) `bool bActive`; (Combat에서만)
- `Tick()` — 현재 회전을 `DesiredAim` 으로 `RotationSpeed` (deg/s) 제한 보간(핵심 메커닉); `bActive` 면 FireRate 주기 자동 발사(라인트레이스 + 빔).

AAutoTurret : AActor

- `FTurretStats Stats`; `AMonster* Target`; `bool bEnabled`; (GameMode가 개수만큼 활성화)
- `Tick()` — `Target` 없거나 죽었으면 최근접 몬스터 재탐색, 자체 회전속도로 추종, 조준 정렬 시 자동 발사.

AMonster : AActor (IDamageable)

- `EMonsterColor Color;` `float HP, MoveSpeed=1000(uu/s); float AttackDPS=1;`
- `FVector TargetLoc;` (기지) `bool bAttacking;`
- `Tick()` — !bAttacking이면 기지로 직진; 도달 시 bAttacking, `Base->TakeHit(AttackDPS*Dt,0)` .
- `TakeHit(Dmg,_)` — HP 감소, 0 → 스파클 FX, `GameMode->OnMonsterKilled()` , Destroy.

AResourceBlock : AActor (IDamageable)

- `EBlockType Type;` `float HP;` `int32 RequiredPower;` `int32 ShardMin/Max;` `int32 ShardValue;` `UMaterialInstanceDynamic* MID;`
- `TakeHit(Dmg, AttackerPower)` — `AttackerPower < RequiredPower` 면 스파크만(피해 0); 아니면 HP 감소, 0 → `DropShards()` 후 Destroy.
- `DropShards()` — `rand(Min..Max)`개 `AResourceShard` 를 임펄스로 흩뿌림(value=ShardValue).

AResourceShard : AActor

- `int32 Value;` `APlayerShip* Homing;` `UStaticMeshComponent* Mesh;`
- `AttractTo(Ship)` — Homing 설정. `Tick()` — Homing이면 가속 보간 이동, 근접 시 `GameMode->AddResources(Value)` + Destroy.

ASpawnPoint : AActor

- `enum class EDir{North,East,South,West}; EDir Direction;` (EditAnywhere)
- `UBillboardComponent` /화살표로 에디터 가시화. GameMode가 방위별로 조회.

ANeonHUD : AHUD

`DrawHUD()` — GameMode 상태 읽어 Canvas로:

- 좌상단 자원 카운트, 웨이브 번호
- 우상단 기지 HP 바(`DrawRect`), $HP \leq 25\%$ 빨강
- Gather: 중앙 "웨이브 N 준비 — [F] 준비 완료", 남은 타이머
- Combat: 잔여 몬스터 수
- Shop: 강화 목록(이름/효과/비용/현재단계), 선택 커서, [$\uparrow \downarrow$ 이동 / Enter 구매 / Tab 다음웨이브]
- GameOver/Victory: 중앙 대형 텍스트 + [R 재시작]

폰트: `GEngine->GetMediumFont()` 등 엔진 기본(에셋 불필요).

APlayerController : APlayerController

- `BeginPlay()` — `UInputMappingContext / UInputAction` 을 코드로 `NewObject` 생성, `IMC->MapKey(IA, Key)` 로 키 매핑, `EnhancedInputSubsystem` 에 IMC 추가 (→ 에디터 입력 에셋 불필요, 전부 CLI 작성 가능).
- 액션: Move(WASD 2D), VerticalThrust(Space/Ctrl 1D), Look(Mouse 2D), Fire(LMB), Ready(F), ShopUp/Down(↑ ↓), ShopConfirm(Enter), NextWave(Tab), Restart(R).
- Combat 중 매 틱 기체 조준 회전을 `AManualTurret::DesiredAim` 에 전달(공존 모델).
- Shop/게임오버 입력은 GameMode 메서드로 라우팅.

데이터 테이블 (코드 인라인 기본값, GameMode UPROPERTY로 튜닝 가능)

블록 (FBlockDef)

Type	색(Emissive)	RequiredPower	HP	ShardMin~Max	ShardValue	필드 비율
Gold	Orange	1	3	5~8	1	60%
Iron	Cyan	2	5	3~5	1	30%
Diamond	White	3	8	10~15	2	10%

필드 총 블록 수 기본 40개(`EditAnywhere`), `SpawnBounds` FBox 내 무작위.

몬스터 / 웨이브 (FWaveDef)

웨이브	입구(방위)	총 수	개체 HP	스폰 간격	MoveSpeed	AttackDPS
1	N	5	1	2.0s	1000uu/s	1
2	N	10	1	1.5s	1000	1
3	N, E	15	2	1.0s	1000	1
4	N, E, S	20	2	1.0s	1000	1
5	N, E, S, W	30	3	0.8s	1000	1

- 다입구는 총 수를 입구에 균등 분배, 입구별 동시 스폰 펌프.
- 색(Orange/Cyan)은 스폰 시 50/50 무작위(기능 동일, 시각 구분만).

강화 (FUpgradeDef) — 단일 상점, 기체+포탑 통합 목록

Id	효과/단계	비용	상한	초기→최대
Ship_Magnet	+0.5m	12	6	2m→5m
Ship_Attack	+1	15	4	1→5
Ship_Speed	+10uu	18	4	40→80
Ship_HP	+2	20	4	3→10(+상한)
Turret_Rotate	+90°/s	20	3	30→90+
Turret_Attack	+1	15	4	—
Turret_FireRate	+1발/s	15	5	—
AutoTurret_Count	+1개	30	4	1→5

단위: 1m = 100uu. 속도 "40 units" → 4000uu/s로 스케일(UE cm 단위), 튜닝 변수로 노출.

구현 우선순위

- 핵심(플레이 가능 최소):** NeonTypes/IDamageable → GameInstance → GameMode 상태머신 → PlayerController(코드 EnhancedInput) → PlayerShip 비행+무기 → Base HP → ResourceBlock+Shard+자석 → AHUD 기본 → Monster+웨이브 스폰 → 클리어/게임오버 루프.
- 부수:** ManualTurret(회전속도 제한) → AutoTurret + 개수 강화 → 상점 UI(선택/구매) → 강화 적용 → 승리/재시작 → 다입구 분배.
- 폴리시:** 네온 MID 색/펄스, FOV 보간, 카메라 세이크, 빔/파괴/사망 파티클(메시 샷드), 기체 잔상 (UNeonTrailComponent), 밸런싱 튜닝.

Day 0 — 사용자 1회 에디터 작업 (CLI 불가, ~15분)

코드 컴파일 성공 후 사용자가 에디터에서:

- 머티리얼** `M_NeonBase` 생성(Content/Materials): Shading Model = Unlit; 파라미터 `BaseColor` (Vector, 기본 회색), `Glow` (Scalar, 기본 3); `BaseColor * Glow` → Emissive Color. (네온의 유일한 필수 에셋. C++가 MID로 색만 제어)
- 레벨** `L_NeonDrift` 생성 + 저장. 어두운 배경(SkyAtmosphere 제거/검정), 약한 Directional.
- 배치: `ABase` (중앙), `ASpawnPoint` ×4(N/E/S/W, Direction 지정), `AAutoTurret` ×4(기지 주변 링), `AManualTurret` (기지 중심), `PlayerStart`.
- Project Settings:** Default GameMode = `ANeonGameMode`, Default Pawn = `APlayerShip`, HUD = `ANeonHUD`, PlayerController = `ANeonPlayerController`, GameInstance = `UNeonGameInstance`; Maps & Modes의 Editor/Game Default Map = `L_NeonDrift`.

Day 1 (~7.5h) — 기본 구조 & 자원 시스템

#	작업	산출 파일	verify
1	타입/인터페이스/스탯 구조체	NeonTypes.h	컴파일 통과
2	GameInstance(파생 스탯)	NeonGameInstance.h/.cpp	컴파일
3	GameMode 상태머신 골격	NeonGameMode.h/.cpp	PIE에서 PreWave→Gather 로그
4	PlayerController + 코드 EnhancedInput	NeonPlayerController.h/.cpp	키 입력 로그
5	PlayerShip 비행(중력·관성·FOV)	PlayerShip.h/.cpp	기체 조종 체감 OK
6	Base HP + 색/플래시	Base.h/.cpp	디버그 피해로 HP 감소·게임오버
7	ResourceBlock 3종 + 공격력 게이트	ResourceBlock.h/.cpp	낮은 공격력은 파괴 불가 확인
8	ResourceShard + 자석 수집	ResourceShard.h/.cpp	블록 파괴→슬래그 흡수→자원 증가
9	AHUD 기본(자원/HP/웨이브)	NeonHUD.h/.cpp	HUD 수치 표시
10	CLI 빌드 + 사용자 PIE 피드백	—	기체 느낌 조정

Day 2 (~8h) — 몬스터 & 포탑

#	작업	산출 파일	verify
1	SpawnPoint 마커	SpawnPoint.h/.cpp	에디터 배치·방위 확인
2	Monster(직진·기지공격·피격사망)	Monster.h/.cpp	기지로 이동·DPS·격추
3	GameMode 웨이브 스폰/클리어 판정	(GameMode 확장)	웨이브1 5마리 클리어
4	AutoTurret(최근접 타겟·연사)	AutoTurret.h/.cpp	자동 격추
5	ManualTurret(회전속도 제한 추종)	ManualTurret.h/.cpp	조준 추종 지연 체감
6	기체무기 ↔ 수동포탑 조준 라우팅	(PC/Ship 확장)	공존 동작
7	빔/머즐 FX(에셋 없는 메시·라인)	(각 발사처)	발사 시각 피드백
8	CLI 빌드 + PIE 웨이브1~3 검증	—	—

Day 3 (~8.5h) — 강화 & 폴리싱

#	작업	산출 파일	verify
1	UpgradeTable + GamelInstance 구매/적용	(GI/GameMode 확장)	구매 후 스탯 변화
2	상점 UI(Canvas 선택/구매/다음웨이브)	(HUD 확장)	키로 강화 선택 동작
3	게임 루프 통합(승리/게임오버/재시작)	(GameMode 확장)	웨이브5 클리어=승리
4	AutoTurret 개수 강화 연동	(GameMode 확장)	강화 시 포탑 활성화 증가
5	폴리시: 네온 펄스·FOV·셰이크·파티클·잔상	NeonTrailComponent 등	시각 확인
6	밸런싱(웨이브/비용/스펙) + 최종 빌드	—	풀 플레이스루

핵심 설계 노트 / 트레이드오프

- **EnhancedInput 코드 생성**: IMC/IA를 `.uasset` 대신 PlayerController에서 `NewObject` + `MapKey` 로 구성 → 에디터 입력 에셋 0, 전부 CLI 작성. (UE 정식 지원 경로)
- **머티리얼 1개만 수동**: 네온 Emissive는 코드만으로 부모 머티리얼을 만들 수 없어 `M_NeonBase` 1개만 사용자 생성. 색/발광은 전부 C++ MID 파라미터로 제어 → 추가 에셋 불필요.
- **단위 스케일**: 기획서 "units"는 UE cm(uu)로 환산해 변수화(예 1m=100uu). 기획 수치는 비율로 유지하고 실제 값은 `EditAnywhere` 로 노출해 Day 3 튜닝.
- **속도 "40 units/sec"**가 너무 느릴 수 있어(=40cm/s) 스케일 계수로 노출, PIE 체감 후 조정.
- **잔상(Trail)**: UE엔 Unity식 Trail Renderer가 없음 → 페이드 메시 에코를 떨구는 경량 `UNeonTrailComponent` 로 대체. Day 3 폴리시, 시간 부족 시 컷 가능.
- **단순화**: DataTable/DataAsset 에셋 대신 코드 인라인 테이블 + GameMode UPROPERTY 노출 (CLI 친화 + 에디터 튜닝 동시 충족). 폴링·고급 AI 등 미요청 기능 미포함(프로토타입).
- **위험요소**: 3일 폴스크래치 + 순수 Canvas UI는 빠듯함. 우선순위 1(핵심 루프)을 먼저 완성해 항상 플레이 가능 상태 유지, 2·3은 시간 내 점진 추가.

검증 (Verification)

컴파일 (CLI에서, 각 작업 후):

```
& "C:/Program Files/Epic Games/UE_5.8/Engine/Build/BatchFiles/Build.bat" `
NEONDRIFTEditor Win64 Development `
-Project="C:/Users/user/Documents/Unreal Projects/NEONDRIFT/NEONDRIFT.uproject" -WaitMutex
```

→ 컴파일 오류 0 확인 후 에디터 PIE 검증.

기능 검증 (에디터 PIE에서, 단계별 체크):

- Day1: 기체 비행/중력/FOV, 블록 공격력 게이트, 슬래그 자석 수집→자원 증가, 기지 HP·게임오버
 - Day2: 웨이브1(5마리) 스폰→격추→클리어, 자동/수동 포탑 동작, 회전속도 제한 체감, 다입구
 - Day3: 상점에서 강화 구매→스탯 반영, 웨이브1~5 풀 플레이, 승리/게임오버/재시작, 폴리시 시각
- 각 Day 종료 시 빌드 통과 + 해당 verify 통과를 완료 기준으로 한다.